

UM6P-inspired Campus

ROS 2 and Gazebo

A beginner's reproducible environment guide

Prepared for Ismaël Assoumane Idi

20 September 2026

This guide concerns environment preparation only. The two quadrotors are static scene placeholders. No physics advancement, flight controller, trajectory, consensus, optimization or numerical campaign is part of this environment.

Contents

1	What you are receiving	2
2	The software, in everyday terms	3
3	Chosen versions and prerequisites	3
4	How the directory is organized	5
5	Build, launch, inspect and stop	6
6	How the scene was constructed	7
7	ROS/Gazebo integration and test results	8
8	Visual Studio Code workflow	10
9	Troubleshooting and safe cleanup	10
10	Provenance, versions and recovery	11
11	Recompile this exact tutorial	11
12	What remains before future flight work	12
	Official references	12
13	Organization and current verification	12
A	Complete first-party source listings	13

1 What you are receiving

You can open a recognizable, photo-inspired campus courtyard containing two parked quadrotors. The package contains the world, all project-specific geometry and materials, a ROS launch file, reusable terminal commands, verification tools, provenance records and this guide’s exact LaTeX source. The software was actually built and loaded with the world paused. The screenshot below came from Gazebo’s screenshot service, not an illustration generator.

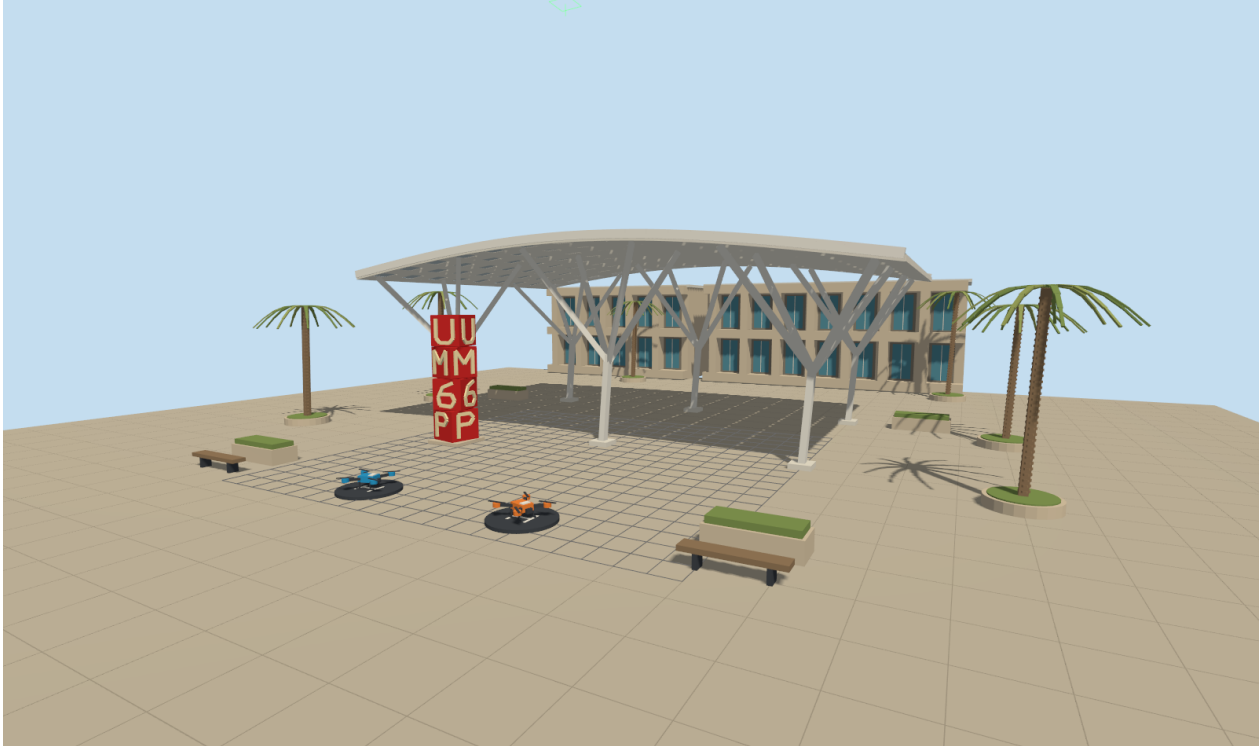


Figure 1: Actual Gazebo Harmonic rendering after the material and shutdown corrections. The blue and orange static quadrotors sit on separate pads. This view emphasizes the lattice and landmark.

This is a recognizable architectural reconstruction, not a surveyed digital twin. Dimensions are assumptions. The four red blocks carry hand-modeled U, M, 6 and P shapes on two visible faces. No official UM6P logo asset is claimed. The original photographs were supplied by the researcher; their hashes are retained, but they are not redistributed because their source and redistribution rights were not supplied.

1.1 Keep the scientific boundary clear

The original campus inspection used Model 0-D v1.4. The current repository context is v1.6, titled *Constraint Consensus-Based Dynamics for Decentralized Path Planning in UAV Swarms*. Scientific documents remain untouched. This decorative scene does not replace the mathematical workspace, obstacles, UAV radii or numerical benchmark.

The later Attempt 3 report supersedes the older two-attempt status: it reports numerical/TCC success and exact saved-plan safety certification, while the exact first-block Sun–Sun oracle failed. Thus the numerical campaign is not fully algorithm-certified. U3/U4 were resolved in the later research context. These are repository findings, not results produced by this environment task. No campaign trajectory is replayed here. Static visual separation does not certify Model 0-D safety.

This guide covers the parked environment only. The separately authorized kinematic demonstrations have their own guide and scenario folders; their existence does not change the paused campus launch.

2 The software, in everyday terms

ROS 2 A set of tools that lets separate programs exchange typed messages and services. Here it starts the environment and receives one informational test message. A running ROS program is called a node.

Gazebo A program for constructing and displaying virtual worlds. It can also compute physics, but this environment intentionally omits the Physics system and stays paused.

ros_gz The official integration packages between ROS 2 and Gazebo. The bridge translates selected message types between their two communication systems.

Workspace A folder where related ROS packages are built and installed together. This delivered folder is the workspace root.

Package A named unit of ROS software or assets with a dependency manifest. Two packages define the parked campus; three more provide the demos.

Launch file A Python configuration that tells ROS which processes to start and which installed files to use. It is not a flight program.

SDF world An XML description of a world and its included models, lighting and systems. SDF means Simulation Description Format. This project does not need URDF.

Mesh and material A mesh is a collection of triangles defining a shape. A material defines its color and surface appearance. OBJ and MTL files store these locally.

Build and install Building prepares the packages. Installing copies their assets and configurations to the workspace’s own install directory and creates the package index used at launch.

Loading a paused world creates the scene and communication endpoints. Rendering produces images using the graphics card. Neither requires time to advance in the physical simulation. Wall time still passes while you inspect a paused scene. This distinction is why the bridge check uses an informational message instead of requiring a continuously advancing clock or camera sensor stream.

3 Chosen versions and prerequisites

Component	Observed in this execution session
OS and architecture	Ubuntu 24.04.5 Noble, x86_64 / amd64
ROS distribution	ROS 2 Jazzy Jalisco, existing apt installation
Gazebo	Harmonic, Sim 8.15.0
Integration	ros_gz 1.0.24 (full Debian revisions recorded)
Graphics	Intel Iris Xe ADL GT2, Mesa 25.2.8, OpenGL 4.6
Build tools	Python 3.12.3, GCC 13.3.0, ament_cmake and colcon

The official compatibility page recommends the Jazzy–Harmonic pairing on Ubuntu 24.04. Jazzy and Harmonic are supported through May 2029 according to the checked official release tables. They are not described here as the newest releases. Jazzy’s official Gazebo vendor packages provide the matching libraries; a second Gazebo repository is unnecessary for this route.

The user’s confirmed target is a Dell Latitude 5430, Intel i5-1245U, 16 GiB RAM and Intel Iris Xe graphics, using Ubuntu 24.04 Noble and Wayland with XWayland. The user supplied older package observations of Gazebo 8.11.0 and ros_gz 1.0.22. Those observations are distinct from this session’s measured 8.15.0 and 1.0.24. The task did not upgrade any installation. Older patch versions were not tested. Recheck the versions in your next local terminal session.

Docker and Podman were not available. The compatible native stack was already installed, so a native isolated workspace is the single maintained route. No container image, digest or dev-container test is claimed. Interactive VS Code execution is documented but was not tested through its UI. Approximately 125 GiB of disk space was available at inspection; the source assets are small, but optional TeX packages can require substantially more space than the campus itself.

3.1 Command conventions

All commands below are for Bash. Lines in a code box are commands unless explicitly labeled as output. Do not type a leading dollar prompt. Replace example paths with your own folder. Quote paths, especially when they contain spaces. Except where a different working directory is stated, run commands from the extracted `ros2_gazebo_um6p_campus` root. Every provided wrapper locates its own root and sources ROS; it does not depend on a previous shell session or edits to your shell startup file.

3.2 Enter and inspect the folder

Prerequisite: the archive has been extracted. Working directory: any Bash terminal. Purpose: enter the delivered workspace and inspect the OS, disk and installed tools.

```
cd "/your/path/ros2_gazebo_um6p_campus"
bash scripts/check_environment.sh
bash scripts/setup.sh --plan
```

The first line changes directory and normally prints nothing. The environment script prints OS, architecture, free space, tool paths and package versions. The setup plan makes no host changes. Observed setup output in this session was:

```
All required packages already installed; no changes.
```

If `/opt/ros/jazzy/setup.bash` is absent, use the fresh Ubuntu setup subsection. If a required package is missing, the plan prints its name. Do not interpret a missing command in an unsourced shell as proof that ROS is not installed.

3.3 Install only missing prerequisites on an existing ROS machine

Prerequisites: Ubuntu 24.04 amd64, the official ROS apt repository already configured, internet access, and your decision to authorize the displayed package-manager transaction. Working directory: workspace root. Purpose: fill missing dependencies without upgrading already installed packages.

```
bash scripts/setup.sh --install-missing
```

Expected result: either the no-change message, or apt's proposed installation followed by completion. The script uses sudo only in this explicit mode. It updates repository metadata and uses `-no-upgrade`. Read apt's proposed changes before accepting. This installation mode was not exercised here because no packages were missing. The underlying list is imported in the appendix.

3.4 Fresh Ubuntu target: official repository setup

This subsection is optional for a new, separately prepared Ubuntu 24.04 amd64 machine; skip it on the confirmed existing laptop installation. Provisioning a clean OS was not tested in this task. Working directory: any terminal for the first block. Prerequisites: administrator access, internet, and a normal Ubuntu installation with the appropriate Noble updates enabled. Purpose: enable Universe and obtain the official ROS apt configuration. These commands change host package sources and require an explicit operator decision.

```
sudo apt-get update
sudo apt-get install curl software-properties-common
sudo add-apt-repository universe
```

The update refreshes the package index, installation supplies the two utilities, and the last command enables Ubuntu's Universe component. Expected result: successful package-manager output; none is claimed as observed here.

The official [ros-apt-source release 1.3.0](#) was checked through GitHub. Working directory: workspace root. Prerequisites: the utilities above, internet and approval for repository configuration. Purpose: download the exact Noble package, verify its published SHA-256, install the repository configuration and then the campus prerequisites.

```
mkdir -p .runtime/setup
curl --fail --location \
  "https://github.com/ros-infrastructure/ros-apt-source/releases/download/1.3.0/ros2-apt-source_1
  .3.0.noble_all.deb" \
  --output .runtime/setup/ros2-apt-source.deb
sha256sum .runtime/setup/ros2-apt-source.deb
```

The first command creates a local download folder; the second retrieves the versioned package; the third prints its checksum. Expected SHA-256 from the official release metadata:

```
f31d84adf5054c7d60ded0e82c0f776a77ea33af53d08f40b9ad7c94cca55296
```

Proceed only if the printed hash agrees. Same working directory and prerequisites; purpose: explicitly install the repository configuration and missing dependencies.

```
sudo dpkg -i .runtime/setup/ros2-apt-source.deb
bash scripts/setup.sh --install-missing
```

Expected result: repository configuration succeeds, then apt proposes any needed runtime packages. No fresh-OS setup, repository modification or host update was executed here. Exact apt dependency availability can change even though this repository-setup package is pinned. If Noble update suites or dependencies conflict, inspect the official ROS installation instructions rather than applying an unreviewed full system upgrade.

4 How the directory is organized

Location	Purpose
README.md	Short entry point and exact commands.
campus_environment/src/um6p_campus_description/	Package manifests, local models, meshes/materials and SDF world.
campus_environment/src/um6p_campus_bringup/	Installed launch, bridge, server and saved camera configuration.
scripts/	Shared entry points; the campus generator is in campus_environment/scripts/.
campus_environment/tests/	Installed asset validation and bounded runtime checks.
.vscode/	Task definitions that call the same shell scripts.
docs/	Assumptions, scene manifest, local target and reproducibility notes.
references/	Repository identities, photo provenance, dependency lists and official sources.
evidence/	Actual logs, JSON results, screenshots and verification record.
tutorial/	Two named PDFs; exact sources and figures under source/.
decisions/	Environment-scope record only; not a mathematical decision.
checkpoints/	Durable status and next actions for recovery.
build/, install/ and log/, .runtime/	Generated local work, ignored for publication.

The folder can later be placed as one new top-level directory in the repository. Nothing in the scientific baseline needs moving. The repository branch was inspected through a pinned connector snapshot at commit `e3a8af05cbb40793f6b48f26ffe042dc51f0ddca`. A terminal Git clone failed private-repository authentication. The delivery is therefore a dedicated environment directory, not a claimed complete Git checkout.

5 Build, launch, inspect and stop

5.1 Build installed packages

Working directory: workspace root. Prerequisites: the dependency plan passes. Purpose: build and install all five project packages locally.

```
bash scripts/build.sh
```

Expected result: colcon reports five packages finished. The current reorganized build completed all five packages successfully. Repeating this command is safe: colcon updates only generated workspace outputs. The script does not use a symlink install; copied installed assets are the runtime source. The package CMake files install their required directories.

5.2 First bounded verification

Working directory: workspace root. Prerequisite: build completed and no other campus test is running in ROS domain 84. Purpose: validate installed resources, then load and close the paused world twice.

```
bash scripts/verify.sh headless
```

Expected result: asset JSON with `status: PASS`, then runtime JSON with `status: PASS`. Evidence is saved under `evidence/local-headless/`. This command has a 120-second overall timeout and bounded sub-operations. It does not open a graphics window. Failure produces a nonzero exit and a result or console error; inspect the saved logs before retrying.

5.3 Open the default camera view

Working directory: workspace root. Prerequisite: build passed and a working graphical session is available. Purpose: open the paused courtyard.

```
bash scripts/launch_environment.sh gui
```

Expected result: Gazebo displays the underside of the pergola, four letter blocks, buildings and both UAVs. The terminal stays occupied while it owns Gazebo and the bridge. There are no play, step, spawn or transform controls in the supplied GUI. Use camera navigation to inspect the scene only. Do not send world-control commands from another client. Gazebo's built-in generic services still exist; the configuration is a workflow boundary, not an adversarial security barrier.

To close the scene, return to the launching terminal and press **Ctrl+C once**. Wait for messages that Gazebo and the bridge finished cleanly. This sends the interrupt to the launch owner, which manages its children. Repeated interrupts or killing the entire process group while ROS also forwards interrupts caused an initial wrapper error during development; the test helper was corrected to signal the owner once.

5.4 Other saved view and automatic visual check

Working directory: workspace root. Prerequisite: the preceding launch has shut down. Purpose of the first command: use the overview camera. Purpose of the second: automatically test two GUI launches and save an actual screenshot. Run them separately, stopping the first with Ctrl+C before the second.

```
bash scripts/launch_environment.sh gui overview
bash scripts/verify.sh gui
```

The overview should reveal more of the full canopy footprint and landscaping. Its configuration is included; the final corrected default camera is the one formally verified and pictured in this guide. Before capturing evidence, the verifier resets the GUI camera to the pose stored in the installed configuration using a camera-only service. It also requires an actual saved PNG. The GUI verification saves under `evidence/local-gui/` and exits automatically. A screenshot-service reply is not enough to establish visual quality: open the PNG and inspect it.

5.5 Direct installed ROS use

For readers who want to understand the wrapper: working directory can be any directory after adjusting the workspace path below. Prerequisites: a successful build and no conflicting launch. Purpose: load the official and project package indexes, then invoke the same installed launch file.

```
source /opt/ros/jazzy/setup.bash
source "/your/path/ros2_gazebo_um6p_campus/install/setup.bash"
export ROS_DOMAIN_ID=84
export ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST
export GZ_PARTITION=ccbo_campus
ros2 launch um6p_campus_bringup campus.launch.py gui:=true
```

The source commands normally print nothing. They make installed package names resolvable. The environment variables restrict ordinary ROS discovery and select a Gazebo partition. The final line opens the same world. Prefer the wrapper for normal use because it also places logs under the workspace. Stop with Ctrl+C. No command in this guide uses Gazebo's run-on-start flag.

6 How the scene was constructed

6.1 Assumptions and dimensions

The paved courtyard is 64 by 60 metres, centered at $(0, 5)$; its visible top is $z = 0$. The main canopy covers $x \in [-14, 14]$ and $y \in [0, 18]$. Its gentle transverse curve is

$$z(x) = 8.6 + 1.35 \cos(\pi x / 28) \quad \text{metres.}$$

This gives 8.6 m at the edges and 9.95 m at the crest. One-metre roof segments approximate the curve. Repeated beams at roughly two-metre spacing form the visible structural lattice. There are six branching support assemblies at $x = -10, 0, 10$ and $y = 3, 15$. Trunks rise from broad feet, then split into four diagonal arms.

Following the researcher's layout feedback, both buildings are translated 12 m farther along +Y, leaving a 13.8 m clear horizontal gap between the pergola edge and the front facade bands. The existing palms, planters, benches, paving and pergola geometry remain unchanged. Two stone-colored rear building volumes are about 16 and 22 m wide, 8 m deep and 9 m high. Their facade columns and horizontal bands stand in front of dark window recesses and blue glass surfaces. This gives real geometric depth without detailed interiors. Six simplified palms, raised planting beds and benches provide restrained landscaping. A directional warm sun supplies shadows. These dimensions are artistic assumptions, not a survey or a load-bearing structural design.

The landmark is at $(-7, 0, 0)$, on a small plinth. Four 1.6 m blocks stack vertically with visible narrow gaps. The generator builds them bottom-up as P, 6, M, U; reading the finished tower top-down yields U, M, 6, P. The blocks alternate +12 and -12 degrees around their vertical axes: U and 6 share one orientation, M and P the other. Each block and its letters rotate together. This follows the additional UM6P photograph supplied in the feedback; angles are artistic estimates. Each letter consists of local polygonal strokes on the front (-Y) and right (+X) faces. This approximates the supplied photograph without introducing an invented official logo file.

6.2 Coordinates, names and UAV poses

World coordinates are right handed: $+X$ points across the courtyard to the right in the design, $+Y$ toward the rear buildings, and $+Z$ upward. Units are metres and radians. There is no GPS datum or surveyed north. SDF poses are written in the order $x, y, z, \text{roll}, \text{pitch}, \text{yaw}$.

Model	Pose	Color
uav_1	$(-3.6, -8.2, 0.12, 0, 0, 0.3)$	Blue
uav_2	$(3.6, -8.2, 0.12, 0, 0, -0.3)$	Orange

Their origins are 7.2 m apart. The pads and UAVs occupy the open forecourt at $y = -8.2$, leaving room for future staging. An unmarked 18 by 18 m area, $x \in [-9, 9]$ and $y \in [-22, -4]$, is kept clear of other above-ground scene objects. This 324 square metre reserve does not assert a flight capacity for 3, 5, 10, 20 or 100 UAVs. Future counts, body dimensions, pad spacing, trajectories and safety requirements must be separately reviewed; no additional vehicle or formation generator is active. The current world still contains exactly two UAVs. Each sits above a 0.12 m display pad. Landing skids start at local $z = 0.005$, approximately 5 mm above the pad top; static geometry preserves this visual clearance without contact solving. Body boxes, four arms, motor cylinders and rigid propeller bars make the vehicles recognizable. The local body center is above the model origin; the listed model pose is not a calibrated center of mass.

The UAV meshes span roughly 1.8 m across including propellers. They are separated from one another, landmark and columns, but no certified safety claim follows. Their small body collision box is a placeholder only. Decorative architecture is visual geometry, not a validated collision environment. No mass, inertia, aerodynamic model or flight controller is validated.

6.3 Generator and resources

The full generator is imported in the appendix. Its `Mesh` helper writes triangles and surface normals for boxes, oriented beams and cylinders. The `Model` helper groups shapes by material, writes local OBJ/MTL files and builds static SDF models. Grouping keeps the scene to 25 meshes and 25,540 triangles rather than thousands of independent entities. The final world includes seven top-level models; only two are UAVs.

Working directory: workspace root. Prerequisite: Python 3 standard library; stop the scene before changing assets. Purpose: regenerate the first-party asset set from its maintained recipe, then reinstall it.

```
python3 campus_environment/scripts/generate_assets.py
bash scripts/build.sh
bash scripts/verify.sh assets
```

Expected result: the generator reports seven static models; build installs them; validation reports PASS. The meshes need no texture images, fonts, Fuel cache or attachment files at runtime. OBJ references local MTL files, while SDF specifies the same colors. A deterministic regeneration hash check is included in the verification record. Changing scene dimensions requires rerunning the relevant asset and visual checks; it does not authorize changing the scientific model.

7 ROS/Gazebo integration and test results

The installed launch uses ament's package index to find resources. It runs Gazebo and a single node named `campus_evidence_bridge`. The bridge is one-way, Gazebo to ROS, on `/campus/evidence`, translating `gz_msgs.StringMsg` to `std_msgs/msg/String`. This informational topic has no connection to UAV movement.

The runtime verifier requests `/world/um6p_campus/scene/info` and checks the seven model names and UAV poses, including yaw quaternions. This is verified service exchange, not just end-point discovery. It subscribes to world statistics and requires every sampled message to report

paused, zero iterations, zero simulation time and no stepping. It also publishes a unique informational string through Gazebo transport and checks that the ROS subscriber receives that exact string. The ROS graph must contain only the bridge and the temporary verifier.

The launch omits Physics, Sensors and UserCommands systems. The only requested server system is SceneBroadcaster. Gazebo may print a default physics-profile name while initializing world metadata; that message does not mean the Physics system was loaded or physics advanced. The zero-iteration evidence and explicit system configuration are the relevant observations.

Check	Outcome and evidence
Two packages build and install	PASS; build log.
World and seven model SDF files	PASS; installed asset validator.
25 local meshes and resource resolution	PASS; asset check and corrected GUI logs.
Exactly two UAVs and specified poses	PASS; scene service responses.
Informational Gazebo-to-ROS message	PASS; exact received strings recorded.
Paused, zero time/iterations, no stepping	PASS in all final sampled statuses.
Headless shutdown and relaunch	PASS; two launches.
Corrected default GUI rendering	PASS; actual PNG visually inspected.
Corrected GUI shutdown and relaunch	PASS; two clean child exits.
Fresh source rebuild in path with spaces	PASS; same installed dependencies, new build/install.
Fresh operating-system provisioning	NOT VERIFIED; no new OS installed.
Docker/dev-container	NOT VERIFIED; runtime absent, route not used.
Older user-reported package versions	NOT VERIFIED.
VS Code task UI	NOT VERIFIED; shared command workflow supplied.
Software-only/EGL rendering	NOT VERIFIED; hardware GUI rendering available.
Physics, sensors, dynamics, flight	OUT OF SCOPE; not executed.

Initial failures and corrected attempts remain documented. The initial GUI material warnings and duplicate-interrupt child exit do not count as clean-shutdown success, even though the early test wrapper returned an overall PASS. The revised verifier checks child logs as well as its parent return code. Ordinary Qt dialog binding warnings remain; the final logs show no missing required assets, Gazebo error or failed child process.

7.1 Portability check

Working directory: workspace root. Prerequisites: installed dependencies and a nonexistent destination outside this folder. Purpose: reproduce the maintained source in a fresh location, including paths with spaces.

```
bash scripts/reproduce_clean.sh "/new/path/campus clean copy"
```

Expected result: new build/install directories, successful asset validation, two paused campus launches and both separately authorized kinematic demo checks. This combined reproduction command advances demo time; it never advances the parked campus world. Existing destinations are

rejected to preserve edits. The current check uses a new directory named `campus organized clean reproduction`, including spaces. It is a clean source rebuild on the same installed OS, not proof of clean-OS package provisioning or old-version compatibility.

8 Visual Studio Code workflow

Open VS Code and choose File > Open Folder, selecting the delivered root. Open Terminal > New Terminal. Use the same setup, build, verification and launch commands shown above. The integrated terminal uses the same installed ROS environment because the scripts source it explicitly.

Alternatively, use Terminal > Run Task and select **Check prerequisites**, **Build**, **Verify campus headless**, or **Campus: launch paused**. The included tasks use process arguments, so workspace paths containing spaces are preserved. The launch terminal remains active. Stop it with Ctrl+C once, then wait for the child exit messages. No dev-container extension is required. This task configuration was inspected; no interactive VS Code UI run is claimed.

9 Troubleshooting and safe cleanup

ROS or Gazebo command not found. Use the supplied scripts, which source the Jazzy installation. If the setup file truly does not exist, use the optional installation route. Do not install a second incompatible distribution simply because a fresh shell is unsourced.

SDF include cannot be found. Gazebo uses `GZ_SIM_RESOURCE_PATH`; the standalone `gz sdf` checker additionally needs `SDF_PATH`. The asset test sets it. Launch from the installed package after building, not by guessing a temporary source path.

Blank window or display unavailable. Check that the command runs inside your graphical desktop session. The execution sandbox initially could not access display :0; authorized host-side rendering worked without changing security settings. Do not disable access controls or change drivers as a first response.

Transport getifaddrs error. The sandbox denied network-interface discovery. The actual tests required permitted local transport access outside it. This is different from a missing ROS package. Do not disable host security settings to guess at a solution.

No bridge message. Do not unpause. Ensure the same partition and domain are used and run one verifier at a time. The verifier publishes its own text independently of physics. A listed topic is not proof of message delivery.

Scene appears unchanged after edits. Rebuild; runtime uses copied installed assets. Close old launch processes before opening the rebuilt scene.

Material warnings. The initial meshes lacked MTL references. The delivered generator writes both OBJ and matching MTL files. Regenerate and rebuild if either is missing; do not download unrelated materials.

Version conflict. Compare local versions with the manifests. Exact apt revisions may no longer be available. Preserve the error and seek a compatible reviewed installation instead of forcing a downgrade or mixing repositories.

GUI wrapper error during shutdown. Press Ctrl+C once in the owner terminal. The final verifier signals its owner once and checks child logs. Do not use a broad command that kills every Gazebo or Python process on the computer.

Working directory: workspace root. Prerequisite: all task-owned launch/test processes have stopped. Purpose: remove only generated workspace products so you can rebuild, retaining source, documentation and evidence.

```
rm -rf -- build install log .runtime
bash scripts/build.sh
```

The deletion is intentionally relative to this workspace; confirm your current folder before using it. Expected result: generated outputs disappear and then build creates new ones. Do not delete your system ROS installation, scientific repository directories or archived evidence. This cleanup was documented, not run against the final working delivery.

10 Provenance, versions and recovery

The source folder records the starting branch commit, Git blob identities, attachment hashes, observed direct package pins, the complete installed package list and file checksums. No credentials or private authentication material are required by the runtime. Original source snapshots were kept outside the publication folder; the manifest links back to their exact repository commit.

A package list is not a permanent reproducibility lock. Apt repositories, TeX packages and website documentation remain mutable. Exact retained binary packages, repository metadata and an OS image would be needed for stronger long-term binary recovery. No container digest is applicable because no container is used. Installation downloads are distinct from runtime: every project-specific runtime asset is already included.

Working directory: extracted workspace root. Prerequisite: retain the delivery's checksum manifest. Purpose: confirm that the delivered files match the recorded bytes.

```
sha256sum --check SHA256SUMS
```

Expected result: OK for every listed file. This identifies the delivered state, not an external signature or a certification of dynamics. The manifest excludes itself and disposable build/cache files. Rebuilds and new local verification folders do not invalidate maintained source hashes; intentional source changes do.

If work must pause, read `checkpoints/STATE.md` first on resumption. Confirm files and dependencies still exist, preserve intervening user edits, and continue from the exact saved next action. Do not assume a cache or process survives. No automatic credit-reset continuation is promised.

11 Recompile this exact tutorial

This PDF is compiled from the delivered LaTeX files. `listings.tex` imports complete first-party scripts and configurations from the actual delivered paths. The generator source replaces enormous repetitive mesh-coordinate listings; all generated geometry is still present as files and hashed.

Working directory: workspace root. Prerequisites: optional document compilation tools. Purpose: install a conventional Ubuntu LaTeX/PDF toolchain if you choose to do so. This optional host installation was not performed here.

```
sudo apt-get install texlive-latex-recommended \
  texlive-latex-extra texlive-fonts-recommended \
  latexmk poppler-utils python3-pil
```

Expected result: a proposed apt transaction, then the tools installed after operator acceptance. Review disk requirements. The task used an available Tectonic executable and a copied cache inside task-owned storage rather than modifying the system TeX installation.

Working directory: workspace root. Prerequisite: latexmk or Tectonic available on PATH. Purpose: regenerate the appendix include list and compile the PDF.

```
bash scripts/build_tutorial.sh
```

Expected result: `tutorial/01_UM6P_Campus_Environment_Setup_and_Coding.pdf` and PDF metadata. Compilation may download TeX support files if using Tectonic without a populated cache; these are documentation-time dependencies only. No scene runtime assets are downloaded.

Working directory: workspace root. Prerequisite: compiled PDF and Poppler. Purpose: render every page for visual inspection.

```
mkdir -p tutorial/qa
pdftoppm -r 110 -png tutorial/01_UM6P_Campus_Environment_Setup_and_Coding.pdf tutorial/qa/page
```

Expected result: one PNG per PDF page. Open every page, check that code wraps within margins, headings and page numbers are readable, and figures and links are present. The final delivery includes a separate PDF QA record. Do not infer layout quality solely from successful compilation.

12 What remains before future flight work

This delivery prepares a paused scene. Future flight experiments require separate authorization and a reviewed design for vehicle dynamics, controllers, state estimation, coordinate alignment, collision geometry, timing, sensors, bridge interfaces and scientific protocol. The parked UAVs must be replaced or extended with validated vehicles. No previous numerical trajectory should be replayed simply because Gazebo now opens.

The researcher authorized publication of the reorganized delivery on 20 September 2026. The final checkpoint, verification record and separate publication receipt identify the handoff state. Scientific repository files outside this delivery remain unchanged.

Official references

1. [Gazebo: Installing Gazebo with ROS](#). Pairing and vendor-package route, rechecked 20 September 2026.
2. [ROS REP 2000](#). Supported platforms and Jazzy lifecycle.
3. [Gazebo release table](#). Harmonic lifecycle.
4. [ROS Jazzy Ubuntu apt installation](#). Web page returned a bot challenge; its official [GitHub documentation source](#) was inspected.
5. [ros_gz_bridge documentation](#) and [official source](#). Installed headers, launch files and message bindings were also inspected.
6. [Gazebo SDF worlds](#). Installed command help and SDF validators provided execution-time checks.

13 Organization and current verification

The campus packages live in `campus_environment/src/`. The two demo packages live below `demo/`; their common code lives below `common/`. The root build script installs all five packages. This guide's source is in `tutorial/source/campus_environment/`; the other guide explains motion. User videos belong to their respective demo media folders. Generated build products are excluded from the source archive.

Current verification is recorded in `evidence/VERIFICATION_CURRENT.md`. Earlier campus evidence is preserved as historical evidence, not silently relabeled. The updated source preserves every campus model and world byte from the approved baseline. GUI pictures must come from actual Gazebo execution. The new organization still resolves models and configuration from installed package paths.

A Complete first-party source listings

These listings import the delivered files directly. Generated mesh coordinates are represented by the complete generator and separately hashed assets.

File 1

.vscode/tasks.json

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "Check prerequisites",
      "type": "process",
      "command": "bash",
      "args": [
        "scripts/setup.sh",
        "--plan"
      ],
      "options": {
        "cwd": "${workspaceFolder}"
      },
      "problemMatcher": []
    },
    {
      "label": "Build",
      "type": "process",
      "command": "bash",
      "args": [
        "scripts/build.sh"
      ],
      "options": {
        "cwd": "${workspaceFolder}"
      },
      "problemMatcher": []
    },
    {
      "label": "Campus: launch paused",
      "type": "process",
      "command": "bash",
      "args": [
        "scripts/launch_environment.sh",
        "gui"
      ],
      "options": {
        "cwd": "${workspaceFolder}"
      },
      "problemMatcher": []
    },
    {
      "label": "Demo 1: launch paused",
      "type": "process",
      "command": "bash",
      "args": [
        "scripts/launch_demo.sh",
        "1",
        "gui"
      ],
      "options": {
        "cwd": "${workspaceFolder}"
      },
      "problemMatcher": []
    }
  ],
}
```

```

{
  "label": "Demo 2: launch paused",
  "type": "process",
  "command": "bash",
  "args": [
    "scripts/launch_demo.sh",
    "2",
    "gui"
  ],
  "options": {
    "cwd": "${workspaceFolder}"
  },
  "problemMatcher": []
},
{
  "label": "Demo: start",
  "type": "process",
  "command": "bash",
  "args": [
    "scripts/demo_control.sh",
    "start"
  ],
  "options": {
    "cwd": "${workspaceFolder}"
  },
  "problemMatcher": []
},
{
  "label": "Demo: pause",
  "type": "process",
  "command": "bash",
  "args": [
    "scripts/demo_control.sh",
    "pause"
  ],
  "options": {
    "cwd": "${workspaceFolder}"
  },
  "problemMatcher": []
},
{
  "label": "Verify campus headless",
  "type": "process",
  "command": "bash",
  "args": [
    "scripts/verify.sh",
    "headless"
  ],
  "options": {
    "cwd": "${workspaceFolder}"
  },
  "problemMatcher": []
}
]
}

```

File 2

campus_environment/scripts/generate_assets.py

```

#!/usr/bin/env python3
"""Deterministic first-party campus meshes. Metres; no downloaded assets."""
from pathlib import Path
from math import sin, cos, pi, sqrt, atan2

```

```

import json
import xml.etree.ElementTree as E

ROOT = Path(__file__).resolve().parents[1]
PKG = ROOT / 'src/um6p_campus_description'
COLORS = {
    'ivory': (.86,.84,.75), 'roof': (.50,.58,.60),
    'stone': (.72,.62,.46), 'recess': (.28,.25,.20),
    'glass': (.10,.30,.39), 'red': (.65,.025,.025),
    'letter': (.98,.83,.48), 'paving': (.60,.55,.44),
    'joint': (.43,.39,.31), 'green': (.23,.34,.10),
    'leaf': (.36,.43,.14), 'trunk': (.31,.21,.12),
    'dark': (.055,.065,.08), 'blue': (.025,.35,.70),
    'orange': (.96,.26,.045), 'white': (.94,.94,.88),
}

def sub(a,b): return tuple(x-y for x,y in zip(a,b))
def cross(a,b): return (a[1]*b[2]-a[2]*b[1],a[2]*b[0]-a[0]*b[2],a[0]*b[1]-a[1]*b[0])
def unit(a):
    d=sqrt(sum(x*x for x in a)); return tuple(x/d for x in a)

class Mesh:
    def __init__(self): self.triangles=[]
    def tri(self,a,b,c): self.triangles.append((a,b,c))
    def box(self,c,s,yaw=0):
        pts=[]
        for x,y,z in [(-1,-1,-1),(1,-1,-1),(1,1,-1),(-1,1,-1),(-1,-1,1),(1,-1,1),(1,1,1),(-1,1,1)]:
            x,y,z=x*s[0]/2,y*s[1]/2,z*s[2]/2
            pts.append((c[0]+cos(yaw)*x-sin(yaw)*y,c[1]+sin(yaw)*x+cos(yaw)*y,c[2]+z))
        for a,b,c_,d in [(0,3,2,1),(4,5,6,7),(0,1,5,4),(1,2,6,5),(2,3,7,6),(3,0,4,7)]:
            self.tri(pts[a],pts[b],pts[c_]); self.tri(pts[a],pts[c_],pts[d])
    def beam(self,a,b,w,depth=None):
        axis=unit(sub(b,a)); ref=(0,0,1) if abs(axis[2])<.95 else (0,1,0)
        u=unit(cross(axis,ref)); v=cross(axis,u); depth=depth or w
        pts=[tuple(p[i]+sx*w/2*u[i]+sy*depth/2*v[i] for i in range(3)) for p in (a,b) for sx,sy in
            [(-1,-1),(1,-1),(1,1),(-1,1)]]
        for a_,b_,c_,d_ in [(0,3,2,1),(4,5,6,7),(0,1,5,4),(1,2,6,5),(2,3,7,6),(3,0,4,7)]:
            self.tri(pts[a_],pts[b_],pts[c_]); self.tri(pts[a_],pts[c_],pts[d_])
    def cylinder(self,c,r,h,n=16):
        for j in range(n):
            a,b=2*pi*j/n,2*pi*(j+1)/n
            p=[(c[0]+r*cos(t),c[1]+r*sin(t),c[2]+z) for z in (-h/2,h/2) for t in (a,b)]
            self.tri(p[0],p[1],p[3]); self.tri(p[0],p[3],p[2])
            self.tri((c[0],c[1],c[2]-h/2),p[1],p[0]); self.tri((c[0],c[1],c[2]+h/2),p[2],p[3])
    def write(self,p):
        out=['# First-party deterministic geometry; units: metres.', f'mtllib {p.stem}.mtl', 'o geometry
            ', 'usemtl surface']
        color=COLORS[p.stem]
        p.with_suffix('.mtl').write_text('newmtl surface\nKa '+ ' '.join(map(str,color))+'\nKd '+ ' '.join
            (map(str,color))+'\nKs 0.08 0.08 0.08\nNs 16\nnd 1\n')
        for i,(a,b,c) in enumerate(self.triangles):
            normal=unit(cross(sub(b,a),sub(c,a)))
            out.extend('v '+ ' '.join(f'{v:.6f}' for v in t) for t in (a,b,c))
            out.append('vn '+ ' '.join(f'{v:.6f}' for v in normal))
            out.append('f '+ ' '.join(f'{3*i+j}/{i+1}' for j in (1,2,3)))
        p.write_text('\n'.join(out)+'\n')

class Model:
    def __init__(self,name,pose=(0,0,0,0,0,0)):
        self.name=name; self.pose=pose; self.mesh={}
    def m(self,color): return self.mesh.setdefault(color,Mesh())
    def save(self):
        folder=PKG/'models'/self.name; (folder/'meshes').mkdir(parents=True,exist_ok=True)
        sdf=E.Element('sdf',version='1.10'); model=E.SubElement(sdf,'model',name=self.name)
        E.SubElement(model,'static').text='true'; link=E.SubElement(model,'link',name='body')

```

```

for color,mesh in sorted(self.mesh.items()):
    mesh.write(folder/'meshes'/f'{color}.obj')
    vis=E.SubElement(link,'visual',name=color); geo=E.SubElement(vis,'geometry')
    E.SubElement(E.SubElement(geo,'mesh'),'uri').text=f'model://{self.name}/meshes/{color}.obj'
    mat=E.SubElement(vis,'material'); rgba=' '.join(map(str,(*COLORS[color],1)))
    E.SubElement(mat,'ambient').text=rgba; E.SubElement(mat,'diffuse').text=rgba
    E.SubElement(mat,'specular').text='.08 .08 .08 1'
if self.name.startswith('uav_'):
    c=E.SubElement(link,'collision',name='placeholder_body')
    E.SubElement(c,'pose').text='0 0 .36 0 0 0'
    E.SubElement(E.SubElement(E.SubElement(c,'geometry'),'box'),'size').text='.65 .45 .24'
E.indent(sdf); E.ElementTree(sdf).write(folder/'model.sdf',encoding='unicode',xml_declaration=
    True)
(folder/'model.config').write_text(f'<model><name>{self.name}</name><version>1.0</version><sdf
    version="1.10">model.sdf</sdf><author><name>CCBO campus preparation</name></author><
    description>Static scene geometry; no validated dynamics.</description></model>\n')

models=[]
def make(name,pose=(0,0,0,0,0,0)):
    m=Model(name,pose); models.append(m); return m

court=make('courtyard')
court.m('paving').box((0,5,-.15),(64,60,.3))
for x in range(-30,31,2): court.m('joint').box((x,5,.001),(.018,58,.002))
for y in range(-23,35,2): court.m('joint').box((0,y,.001),(62,.018,.002))
for x in (-3.6,3.6):
    court.m('dark').cylinder((x,-8.2,.06),1.2,.12,48)
    court.m('white').box((x,-8.2,.122),(.8,.1,.004))
    for dx in (-.35,.35): court.m('white').box((x+dx,-8.2,.122),(.10,.75,.004))

pergola=make('pergola')
def roof_z(x): return 8.6+1.35*cos(pi*x/28)
for x in range(-14,14):
    for y in range(0,18,2):
        pergola.m('roof').beam((x+.04,y+1,roof_z(x)+.17),(x+.96,y+1,roof_z(x+1)+.17),1.91,.09)
for x in range(-14,15,2): pergola.m('ivory').beam((x,0,roof_z(x)),(x,18,roof_z(x)),.12,.24)
for y in range(0,19,2):
    for x in range(-14,14): pergola.m('ivory').beam((x,y,roof_z(x)),(x+1,y,roof_z(x+1)),.15,.25)
for y in (0,18):
    for x in range(-14,14): pergola.m('white').beam((x,y,roof_z(x)+.16),(x+1,y,roof_z(x+1)+.16),
        .28,.38)
for x in (-10,0,10):
    for y in (3,15):
        pergola.m('white').beam((x,y,0),(x+.35,y,4.6),.30,.40)
        for dx,dy in [(-3,-2.5),(3,-2.5),(-3,2.5),(3,2.5)]:
            pergola.m('white').beam((x+.35,y,4.2),(x+dx,y+dy,roof_z(x+dx)-.15),.22,.28)
        pergola.m('ivory').box((x,y,.12),(1,1,.24))

building=make('campus_buildings',(0,12,0,0,0,0))
# Columns and floor bands leave real recesses ahead of the glass backing.
for cx,width in [(-16,16),(5,22)]:
    building.m('stone').box((cx,25,4.5),(width,8,9))
    for z in (.5,4.6,8.7): building.m('stone').box((cx,20.4,z),(width,1.2,.8))
    for x in range(int(cx-width/2)+1,int(cx+width/2),3):
        building.m('stone').box((x,20.4,4.6),(.7,1.2,7.4))
        for z in (2.5,6.6):
            building.m('recess').box((x+1.35,20.98,z),(1.9,.04,3.2))
            building.m('glass').box((x+1.35,20.94,z),(1.45,.035,2.9))
            building.m('ivory').box((x+1.35,20.90,z),(.05,.04,2.9))
        building.m('ivory').box((cx,24.8,9.12),(width+.5,8.6,.24))

landmark=make('um6p_landmark',(-7,0,0,0,0,0))
# Hand-drawn line letters on front (-Y) and right (+X) faces. No font/logo asset.
letters={
    'U': [((-38,.48),(-38,-.30),(-.22,-.48),(.22,-.48),(.38,-.30),(.38,.48))],

```

```

'M': [((-40,-.48),(-40,.48),(0,-.05),(40,.48),(40,-.48))],
'6': [((-30,.48),(-.15,.30),(-.36,-.12),(-.30,-.43),(22,-.48),(38,-.24),(25,.02),(-.31,.02))],
'P': [((-35,-.48),(-.35,.48),(20,.48),(38,.28),(20,.05),(-.35,.05))],
}
for i,ch in enumerate('P6MU'):
    starts={c:len(m.triangles) for c,m in landmark.mesh.items()}
    z=.18+.79+i*1.60
    landmark.m('red').box((0,0,z),(1.6,1.6,1.56),0)
    for face in ('front','right'):
        for line in letters[ch]:
            for a,b in zip(line,line[1:]):
                a=(a[0]*1.25,a[1]*1.25); b=(b[0]*1.25,b[1]*1.25)
                p=(a[0],-.81,z+a[1]) if face=='front' else (.81,a[0],z+a[1])
                q=(b[0],-.81,z+b[1]) if face=='front' else (.81,b[0],z+b[1])
                landmark.m('letter').beam(p,q,.17,.17)
    # Rotate each block and its lettering together; U/6 share +12 deg, M/P -12 deg.
    angle=(12 if ch in 'U6' else -12)*pi/180
    for color,mesh in landmark.mesh.items():
        start=starts.get(color,0)
        mesh.triangles[start:]=[tuple((cos(angle)*x-sin(angle)*y,sin(angle)*x+cos(angle)*y,z_) for x,y,
            z_ in tri) for tri in mesh.triangles[start:]]
    landmark.m('stone').box((0,0,.09),(2,2,.18))

landscape=make('landscaping')
for j,(x,y) in enumerate([(-19,0),(18,1),(-19,11),(18,12),(-12,29),(15,29)]):
    h=7.0+(j%3)*.55
    landscape.m('stone').cylinder((x,y,.2),1.25,.4,24)
    landscape.m('green').cylinder((x,y,.41),1.10,.08,24)
    landscape.m('trunk').beam((x,y,.4),(x+.3,y,h),.30)
    for z in range(1,int(h*5)):
        landscape.m('trunk').cylinder((x+.3*z/(h*5),y,z*.2),.19,.08,10)
    for k in range(11):
        a=k*2*pi/11+j*.22
        p=(x+.3,y,h)
        for t in range(5):
            r1,r2=t*.55,(t+1)*.55
            q=(x+.3+r2*cos(a),y+r2*sin(a),h+.5*sin(pi*r2/2.75)-.45*r2)
            landscape.m('leaf' if k%2 else 'green').beam(p,q,.28*(1-t/6),.06); p=q
    for x,y in [(-12,-7),(11,-7),(-15,14),(14,15)]:
        landscape.m('stone').box((x,y,.35),(3,1.2,.7))
        landscape.m('green').box((x,y,.75),(2.8,1,.35))
    for x in (-12,11):
        landscape.m('trunk').box((x,-9,.55),(3,.55,.16))
        for dx in (-1,1): landscape.m('dark').box((x+dx,-9,.27),(1.5,.4,.54))

for i,(x,color,yaw) in enumerate([(-3.6,'blue',.3),(3.6,'orange',-.3)],1):
    m=make(f'uav_{i}',(x,-8.2,.12,0,0,yaw))
    m.m(color).box((0,0,.38),(65,.45,.24))
    m.m('white').box((.16,0,.51),(18,.30,.04))
    m.m('dark').box((.35,0,.30),(16,.22,.16))
    for sx in (-1,1):
        for sy in (-1,1):
            p=(sx*.58,sy*.58,.42)
            m.m('dark').beam((sx*.2,sy*.15,.38),p,.10)
            m.m(color).cylinder(p,.13,.14)
            m.m('dark').box((p[0],p[1],.51),(62,.075,.025),sx*sy*.4)
    for y in (-.27,.27):
        m.m('dark').beam((-36,y,.03),(36,y,.03),.05)
        for x_ in (-.22,.22): m.m('dark').beam((x_,y,.03),(x_,y,.30),.045)

for m in models: m.save()
world=E.Element('sdf',version='1.10'); w=E.SubElement(world,'world',name='um6p_campus')
# Explicit rendering-only system list prevents automatic default Physics loading.
E.SubElement(w,'plugin',filename='gz-sim-scene-broadcaster-system',name='gz::sim::systems::
    SceneBroadcaster')

```

```

scene=E.SubElement(w,'scene'); E.SubElement(scene,'ambient').text='.65 .65 .65 1'
E.SubElement(scene,'background').text='.55 .72 .86 1'; E.SubElement(scene,'shadows').text='true'
light=E.SubElement(w,'light',name='warm_sun',type='directional')
for k,v in {'pose':'0 0 20 0 0 0','diffuse':'.95 .86 .72 1','specular':'.2 .2 .2 1','direction
':'.45 .5 -.85','cast_shadows':'true'}.items(): E.SubElement(light,k).text=v
for m in models:
    inc=E.SubElement(w,'include'); E.SubElement(inc,'uri').text=f'model://{m.name}'
    E.SubElement(inc,'name').text=m.name; E.SubElement(inc,'pose').text=' '.join(map(str,m.pose))
(PKG/'worlds').mkdir(exist_ok=True); E.indent(world)
E.ElementTree(world).write(PKG/'worlds/campus.sdf',encoding='unicode',xml_declaration=True)
(ROOT/'docs/scene-manifest.json').write_text(json.dumps({
    'units':'metres, radians','frame':'right-handed; X courtyard right, Y toward rear buildings, Z up
    ; no georeference',
    'models':[{ 'name':m.name,'pose':m.pose,'static':True,'triangles':sum(len(x.triangles) for x in m.
    mesh.values())} for m in models],
    'pergola_dimensions':[28,18],'canopy_height_range':[8.6,9.95],
    'landmark_blocks':4,'landmark_top_to_bottom':'UM6P','uav_center_spacing':7.2,
    'landmark_block_yaw_degrees_top_to_bottom':[12,-12,12,-12],
    'building_translation_y':12,'pergola_to_front_facade_gap':13.8,
    'reserved_forecourt':{'x':[-9,9],'y':[-22,-4],'area_m2':324,'status':'unmarked clear staging
    reserve; not a certified flight envelope or capacity claim'},
    'physics_plugin':False,'assets':'first-party procedural OBJ; no external runtime assets'},indent
    =2)+'\n')
print('Generated',len(models),'static models; exactly uav_1 and uav_2.')

```

File 3

campus_environment/src/um6p_campus_bringup/CMakeLists.txt

```

cmake_minimum_required(VERSION 3.16)
project(um6p_campus_bringup)
find_package(ament_cmake REQUIRED)
install(DIRECTORY launch config DESTINATION share/${PROJECT_NAME})
ament_package()

```

File 4

campus_environment/src/um6p_campus_bringup/config/bridge.yaml

```

# A read-only informational channel. Tests publish text from Gazebo transport.
- ros_topic_name: /campus/evidence
  gz_topic_name: /campus/evidence
  ros_type_name: std_msgs/msg/String
  gz_type_name: gz.msgs.StringMsg
  direction: GZ_TO_ROS
  lazy: false

```

File 5

campus_environment/src/um6p_campus_bringup/config/gui.config

```

<?xml version="1.0"?>
<window><width>1600</width><height>1000</height>
  <menus><drawer default="false"/></menus>
  <dialog_on_exit>false</dialog_on_exit>
</window>
<plugin filename="MinimalScene" name="Campus | PAUSED environment preparation">
  <gz-gui><title>Campus | PAUSED environment preparation</title>
    <property type="bool" key="showTitleBar">false</property>
    <property type="string" key="state">docked</property></gz-gui>
  <engine>ogre2</engine><scene>scene</scene>
  <ambient_light>.65 .65 .65</ambient_light>

```

```

    <background_color>.55 .72 .86</background_color>
    <camera_pose>16 -24 6.8 0 0.10 2.06</camera_pose>
</plugin>
<plugin filename="GzSceneManager" name="Scene manager">
    <gz-gui><property key="state" type="string">floating</property>
    <property key="showTitleBar" type="bool">>false</property>
    <property key="width" type="double">5</property>
    <property key="height" type="double">5</property></gz-gui>
</plugin>
<plugin filename="InteractiveViewControl" name="Camera inspection">
    <gz-gui><property key="state" type="string">floating</property>
    <property key="showTitleBar" type="bool">>false</property>
    <property key="width" type="double">5</property>
    <property key="height" type="double">5</property></gz-gui>
</plugin>
<plugin filename="Screenshot" name="Screenshot">
    <gz-gui><property key="state" type="string">floating</property>
    <property key="showTitleBar" type="bool">>false</property>
    <property key="width" type="double">50</property>
    <property key="height" type="double">50</property></gz-gui>
</plugin>

<plugin filename="CameraTracking" name="Saved camera views">
    <gz-gui><property key="state" type="string">floating</property>
    <property key="showTitleBar" type="bool">>false</property>
    <property key="width" type="double">5</property>
    <property key="height" type="double">5</property></gz-gui>
</plugin>

```

File 6

campus_environment/src/um6p_campus_bringup/config/overview.config

```

<?xml version="1.0"?>
<window><width>1600</width><height>1000</height>
  <menus><drawer default="false"/></menus>
  <dialog_on_exit>>false</dialog_on_exit>
</window>
<plugin filename="MinimalScene" name="Campus | PAUSED environment preparation">
  <gz-gui><title>Campus | PAUSED environment preparation</title>
  <property type="bool" key="showTitleBar">>false</property>
  <property type="string" key="state">docked</property></gz-gui>
  <engine>ogre2</engine><scene>scene</scene>
  <ambient_light>.65 .65 .65</ambient_light>
  <background_color>.55 .72 .86</background_color>
  <camera_pose>19 -25 13 0 0.25 2.08</camera_pose>
</plugin>
<plugin filename="GzSceneManager" name="Scene manager">
  <gz-gui><property key="state" type="string">floating</property>
  <property key="showTitleBar" type="bool">>false</property>
  <property key="width" type="double">5</property>
  <property key="height" type="double">5</property></gz-gui>
</plugin>
<plugin filename="InteractiveViewControl" name="Camera inspection">
  <gz-gui><property key="state" type="string">floating</property>
  <property key="showTitleBar" type="bool">>false</property>
  <property key="width" type="double">5</property>
  <property key="height" type="double">5</property></gz-gui>
</plugin>
<plugin filename="Screenshot" name="Screenshot">
  <gz-gui><property key="state" type="string">floating</property>
  <property key="showTitleBar" type="bool">>false</property>
  <property key="width" type="double">50</property>
  <property key="height" type="double">50</property></gz-gui>

```

```

</plugin>

<plugin filename="CameraTracking" name="Saved camera views">
  <gz-gui><property key="state" type="string">floating</property>
  <property key="showTitleBar" type="bool">false</property>
  <property key="width" type="double">5</property>
  <property key="height" type="double">5</property></gz-gui>
</plugin>

```

File 7

campus_environment/src/um6p_campus_bringup/config/server.config

```

<?xml version="1.0"?>
<server_config><plugins>
  <plugin entity_name="*" entity_type="world"
    filename="gz-sim-scene-broadcaster-system"
    name="gz::sim::systems::SceneBroadcaster"/>
</plugins></server_config>

```

File 8

campus_environment/src/um6p_campus_bringup/launch/campus.launch.py

```

"""Rendering-only world. No run flag, physics system or control bridge."""
from pathlib import Path
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, ExecuteProcess, OpaqueFunction,
    RegisterEventHandler, EmitEvent
from launch.event_handlers import OnProcessExit
from launch.events import Shutdown
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def start(context):
    desc = Path(get_package_share_directory('um6p_campus_description'))
    bringup = Path(get_package_share_directory('um6p_campus_bringup'))
    view = LaunchConfiguration('view').perform(context)
    gui = LaunchConfiguration('gui').perform(context).lower() == 'true'
    command = ['gz', 'sim', '-v', '3', str(desc / 'worlds/campus.sdf')]
    if gui:
        command += ['--gui-config', str(bringup / 'config' / ('overview.config' if view == '
overview' else 'gui.config'))]
    else:
        command += ['-s']
    gazebo = ExecuteProcess(cmd=command, output='screen',
        additional_env={'GZ_SIM_RESOURCE_PATH': str(desc / 'models'),
            'GZ_SIM_SERVER_CONFIG_PATH': str(bringup / 'config/server.config')})
    bridge = Node(package='ros_gz_bridge', executable='parameter_bridge',
        name='campus_evidence_bridge', output='screen',
        parameters=[{'config_file': str(bringup / 'config/bridge.yaml')}])
    return [gazebo, bridge, RegisterEventHandler(OnProcessExit(
        target_action=gazebo, on_exit=[EmitEvent(event=Shutdown(reason='Gazebo exited'))])

def generate_launch_description():
    return LaunchDescription([
        DeclareLaunchArgument('gui', default_value='true', choices=['true', 'false']),
        DeclareLaunchArgument('view', default_value='courtyard', choices=['courtyard', 'overview
']),
        OpaqueFunction(function=start)])

```

File 9

campus_environment/src/um6p_campus_bringup/package.xml

```
<?xml version="1.0"?>
<package format="3">
  <name>um6p_campus_bringup</name>
  <version>1.0.0</version>
  <description>Paused UM6P-inspired campus environment preparation.</description>
  <maintainer email="ismo-idi@users.noreply.github.com">Ismael Assoumane Idi</maintainer>
  <license>UNLICENSED</license>
  <buildtool_depend>ament_cmake</buildtool_depend>
  <exec_depend>um6p_campus_description</exec_depend>
  <exec_depend>launch</exec_depend>
  <exec_depend>launch_ros</exec_depend>
  <exec_depend>ament_index_python</exec_depend>
  <exec_depend>ros_gz_sim</exec_depend>
  <exec_depend>ros_gz_bridge</exec_depend>
  <exec_depend>std_msgs</exec_depend>
  <export><build_type>ament_cmake</build_type></export>
</package>
```

File 10

campus_environment/src/um6p_campus_description/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(um6p_campus_description)
find_package(ament_cmake REQUIRED)
install(DIRECTORY models worlds DESTINATION share/${PROJECT_NAME})
ament_package()
```

File 11

campus_environment/src/um6p_campus_description/models/campus_buildings/model.config

```
<model><name>campus_buildings</name><version>1.0</version><sdf version="1.10">model.sdf</sdf><
  author><name>CCBO campus preparation</name></author><description>Static scene geometry; no
  validated dynamics.</description></model>
```

File 12

campus_environment/src/um6p_campus_description/models/campus_buildings/model.sdf

```
<?xml version='1.0' encoding='utf-8'?>
<sdf version="1.10">
  <model name="campus_buildings">
    <static>true</static>
    <link name="body">
      <visual name="glass">
        <geometry>
          <mesh>
            <uri>model://campus_buildings/meshes/glass.obj</uri>
          </mesh>
        </geometry>
        <material>
          <ambient>0.1 0.3 0.39 1</ambient>
          <diffuse>0.1 0.3 0.39 1</diffuse>
          <specular>.08 .08 .08 1</specular>
        </material>
      </visual>
      <visual name="ivory">
        <geometry>
```

```

    <mesh>
      <uri>model://campus_buildings/meshes/ivory.obj</uri>
    </mesh>
  </geometry>
</material>
  <ambient>0.86 0.84 0.75 1</ambient>
  <diffuse>0.86 0.84 0.75 1</diffuse>
  <specular>.08 .08 .08 1</specular>
</material>
</visual>
<visual name="recess">
  <geometry>
    <mesh>
      <uri>model://campus_buildings/meshes/recess.obj</uri>
    </mesh>
  </geometry>
  <material>
    <ambient>0.28 0.25 0.2 1</ambient>
    <diffuse>0.28 0.25 0.2 1</diffuse>
    <specular>.08 .08 .08 1</specular>
  </material>
</visual>
<visual name="stone">
  <geometry>
    <mesh>
      <uri>model://campus_buildings/meshes/stone.obj</uri>
    </mesh>
  </geometry>
  <material>
    <ambient>0.72 0.62 0.46 1</ambient>
    <diffuse>0.72 0.62 0.46 1</diffuse>
    <specular>.08 .08 .08 1</specular>
  </material>
</visual>
</link>
</model>
</sdf>

```

File 13

campus_environment/src/um6p_campus_description/models/courtyard/model.config

```

<model><name>courtyard</name><version>1.0</version><sdf version="1.10">model.sdf</sdf><author><
  name>CCBO campus preparation</name></author><description>Static scene geometry; no validated
  dynamics.</description></model>

```

File 14

campus_environment/src/um6p_campus_description/models/courtyard/model.sdf

```

<?xml version='1.0' encoding='utf-8'?>
<sdf version="1.10">
  <model name="courtyard">
    <static>true</static>
    <link name="body">
      <visual name="dark">
        <geometry>
          <mesh>
            <uri>model://courtyard/meshes/dark.obj</uri>
          </mesh>
        </geometry>
        <material>
          <ambient>0.055 0.065 0.08 1</ambient>

```

```

        <diffuse>0.055 0.065 0.08 1</diffuse>
        <specular>.08 .08 .08 1</specular>
    </material>
</visual>
<visual name="joint">
    <geometry>
        <mesh>
            <uri>model://courtyard/meshes/joint.obj</uri>
        </mesh>
    </geometry>
    <material>
        <ambient>0.43 0.39 0.31 1</ambient>
        <diffuse>0.43 0.39 0.31 1</diffuse>
        <specular>.08 .08 .08 1</specular>
    </material>
</visual>
<visual name="paving">
    <geometry>
        <mesh>
            <uri>model://courtyard/meshes/paving.obj</uri>
        </mesh>
    </geometry>
    <material>
        <ambient>0.6 0.55 0.44 1</ambient>
        <diffuse>0.6 0.55 0.44 1</diffuse>
        <specular>.08 .08 .08 1</specular>
    </material>
</visual>
<visual name="white">
    <geometry>
        <mesh>
            <uri>model://courtyard/meshes/white.obj</uri>
        </mesh>
    </geometry>
    <material>
        <ambient>0.94 0.94 0.88 1</ambient>
        <diffuse>0.94 0.94 0.88 1</diffuse>
        <specular>.08 .08 .08 1</specular>
    </material>
</visual>
</link>
</model>
</sdf>

```

File 15

campus_environment/src/um6p_campus_description/models/landscaping/model.config

```

<model><name>landscaping</name><version>1.0</version><sdf version="1.10">model.sdf</sdf><author><
  name>CCBO campus preparation</name></author><description>Static scene geometry; no validated
  dynamics.</description></model>

```

File 16

campus_environment/src/um6p_campus_description/models/landscaping/model.sdf

```

<?xml version='1.0' encoding='utf-8'?>
<sdf version="1.10">
  <model name="landscaping">
    <static>true</static>
    <link name="body">
      <visual name="dark">
        <geometry>

```

```

    <mesh>
      <uri>model://landscaping/meshes/dark.obj</uri>
    </mesh>
  </geometry>
</material>
<material>
  <ambient>0.055 0.065 0.08 1</ambient>
  <diffuse>0.055 0.065 0.08 1</diffuse>
  <specular>.08 .08 .08 1</specular>
</material>
</visual>
<visual name="green">
  <geometry>
    <mesh>
      <uri>model://landscaping/meshes/green.obj</uri>
    </mesh>
  </geometry>
  <material>
    <ambient>0.23 0.34 0.1 1</ambient>
    <diffuse>0.23 0.34 0.1 1</diffuse>
    <specular>.08 .08 .08 1</specular>
  </material>
</visual>
<visual name="leaf">
  <geometry>
    <mesh>
      <uri>model://landscaping/meshes/leaf.obj</uri>
    </mesh>
  </geometry>
  <material>
    <ambient>0.36 0.43 0.14 1</ambient>
    <diffuse>0.36 0.43 0.14 1</diffuse>
    <specular>.08 .08 .08 1</specular>
  </material>
</visual>
<visual name="stone">
  <geometry>
    <mesh>
      <uri>model://landscaping/meshes/stone.obj</uri>
    </mesh>
  </geometry>
  <material>
    <ambient>0.72 0.62 0.46 1</ambient>
    <diffuse>0.72 0.62 0.46 1</diffuse>
    <specular>.08 .08 .08 1</specular>
  </material>
</visual>
<visual name="trunk">
  <geometry>
    <mesh>
      <uri>model://landscaping/meshes/trunk.obj</uri>
    </mesh>
  </geometry>
  <material>
    <ambient>0.31 0.21 0.12 1</ambient>
    <diffuse>0.31 0.21 0.12 1</diffuse>
    <specular>.08 .08 .08 1</specular>
  </material>
</visual>
</link>
</model>
</sdf>

```

File 17

campus_environment/src/um6p_campus_description/models/pergola/model.config

```
<model><name>pergola</name><version>1.0</version><sdf version="1.10">model.sdf</sdf><author><name>
  CCBO campus preparation</name></author><description>Static scene geometry; no validated
  dynamics.</description></model>
```

File 18

campus_environment/src/um6p_campus_description/models/pergola/model.sdf

```
<?xml version='1.0' encoding='utf-8'?>
<sdf version="1.10">
  <model name="pergola">
    <static>true</static>
    <link name="body">
      <visual name="ivory">
        <geometry>
          <mesh>
            <uri>model://pergola/meshes/ivory.obj</uri>
          </mesh>
        </geometry>
        <material>
          <ambient>0.86 0.84 0.75 1</ambient>
          <diffuse>0.86 0.84 0.75 1</diffuse>
          <specular>.08 .08 .08 1</specular>
        </material>
      </visual>
      <visual name="roof">
        <geometry>
          <mesh>
            <uri>model://pergola/meshes/roof.obj</uri>
          </mesh>
        </geometry>
        <material>
          <ambient>0.5 0.58 0.6 1</ambient>
          <diffuse>0.5 0.58 0.6 1</diffuse>
          <specular>.08 .08 .08 1</specular>
        </material>
      </visual>
      <visual name="white">
        <geometry>
          <mesh>
            <uri>model://pergola/meshes/white.obj</uri>
          </mesh>
        </geometry>
        <material>
          <ambient>0.94 0.94 0.88 1</ambient>
          <diffuse>0.94 0.94 0.88 1</diffuse>
          <specular>.08 .08 .08 1</specular>
        </material>
      </visual>
    </link>
  </model>
</sdf>
```

File 19

campus_environment/src/um6p_campus_description/models/uav_1/model.config

```
<model><name>uav_1</name><version>1.0</version><sdf version="1.10">model.sdf</sdf><author><name>
  CCBO campus preparation</name></author><description>Static scene geometry; no validated
  dynamics.</description></model>
```

File 20

campus_environment/src/um6p_campus_description/models/uav_1/model.sdf

```
<?xml version='1.0' encoding='utf-8'?>
<sdf version="1.10">
  <model name="uav_1">
    <static>true</static>
    <link name="body">
      <visual name="blue">
        <geometry>
          <mesh>
            <uri>model://uav_1/meshes/blue.obj</uri>
          </mesh>
        </geometry>
        <material>
          <ambient>0.025 0.35 0.7 1</ambient>
          <diffuse>0.025 0.35 0.7 1</diffuse>
          <specular>.08 .08 .08 1</specular>
        </material>
      </visual>
      <visual name="dark">
        <geometry>
          <mesh>
            <uri>model://uav_1/meshes/dark.obj</uri>
          </mesh>
        </geometry>
        <material>
          <ambient>0.055 0.065 0.08 1</ambient>
          <diffuse>0.055 0.065 0.08 1</diffuse>
          <specular>.08 .08 .08 1</specular>
        </material>
      </visual>
      <visual name="white">
        <geometry>
          <mesh>
            <uri>model://uav_1/meshes/white.obj</uri>
          </mesh>
        </geometry>
        <material>
          <ambient>0.94 0.94 0.88 1</ambient>
          <diffuse>0.94 0.94 0.88 1</diffuse>
          <specular>.08 .08 .08 1</specular>
        </material>
      </visual>
      <collision name="placeholder_body">
        <pose>0 0 .36 0 0 0</pose>
        <geometry>
          <box>
            <size>.65 .45 .24</size>
          </box>
        </geometry>
      </collision>
    </link>
  </model>
</sdf>
```

File 21

campus_environment/src/um6p_campus_description/models/uav_2/model.config

```
<model><name>uav_2</name><version>1.0</version><sdf version="1.10">model.sdf</sdf><author><name>
CCBO campus preparation</name></author><description>Static scene geometry; no validated
dynamics.</description></model>
```

File 22

campus_environment/src/um6p_campus_description/models/uav_2/model.sdf

```
<?xml version='1.0' encoding='utf-8'?>
<sdf version="1.10">
  <model name="uav_2">
    <static>true</static>
    <link name="body">
      <visual name="dark">
        <geometry>
          <mesh>
            <uri>model://uav_2/meshes/dark.obj</uri>
          </mesh>
        </geometry>
        <material>
          <ambient>0.055 0.065 0.08 1</ambient>
          <diffuse>0.055 0.065 0.08 1</diffuse>
          <specular>.08 .08 .08 1</specular>
        </material>
      </visual>
      <visual name="orange">
        <geometry>
          <mesh>
            <uri>model://uav_2/meshes/orange.obj</uri>
          </mesh>
        </geometry>
        <material>
          <ambient>0.96 0.26 0.045 1</ambient>
          <diffuse>0.96 0.26 0.045 1</diffuse>
          <specular>.08 .08 .08 1</specular>
        </material>
      </visual>
      <visual name="white">
        <geometry>
          <mesh>
            <uri>model://uav_2/meshes/white.obj</uri>
          </mesh>
        </geometry>
        <material>
          <ambient>0.94 0.94 0.88 1</ambient>
          <diffuse>0.94 0.94 0.88 1</diffuse>
          <specular>.08 .08 .08 1</specular>
        </material>
      </visual>
      <collision name="placeholder_body">
        <pose>0 0 .36 0 0 0</pose>
        <geometry>
          <box>
            <size>.65 .45 .24</size>
          </box>
        </geometry>
      </collision>
    </link>
  </model>
</sdf>
```

File 23

campus_environment/src/um6p_campus_description/models/um6p_landmark/model.config

```
<model><name>um6p_landmark</name><version>1.0</version><sdf version="1.10">model.sdf</sdf><author>
  <name>CCBO campus preparation</name></author><description>Static scene geometry; no validated
  dynamics.</description></model>
```

File 24

campus_environment/src/um6p_campus_description/models/um6p_landmark/model.sdf

```
<?xml version='1.0' encoding='utf-8'?>
<sdf version="1.10">
  <model name="um6p_landmark">
    <static>true</static>
    <link name="body">
      <visual name="letter">
        <geometry>
          <mesh>
            <uri>model://um6p_landmark/meshes/letter.obj</uri>
          </mesh>
        </geometry>
        <material>
          <ambient>0.98 0.83 0.48 1</ambient>
          <diffuse>0.98 0.83 0.48 1</diffuse>
          <specular>.08 .08 .08 1</specular>
        </material>
      </visual>
      <visual name="red">
        <geometry>
          <mesh>
            <uri>model://um6p_landmark/meshes/red.obj</uri>
          </mesh>
        </geometry>
        <material>
          <ambient>0.65 0.025 0.025 1</ambient>
          <diffuse>0.65 0.025 0.025 1</diffuse>
          <specular>.08 .08 .08 1</specular>
        </material>
      </visual>
      <visual name="stone">
        <geometry>
          <mesh>
            <uri>model://um6p_landmark/meshes/stone.obj</uri>
          </mesh>
        </geometry>
        <material>
          <ambient>0.72 0.62 0.46 1</ambient>
          <diffuse>0.72 0.62 0.46 1</diffuse>
          <specular>.08 .08 .08 1</specular>
        </material>
      </visual>
    </link>
  </model>
</sdf>
```

File 25

campus_environment/src/um6p_campus_description/package.xml

```
<?xml version="1.0"?>
<package format="3">
  <name>um6p_campus_description</name>
  <version>1.0.0</version>
  <description>Paused UM6P-inspired campus environment preparation.</description>
  <maintainer email="ismo-idi@users.noreply.github.com">Ismael Assoumane Idi</maintainer>
  <license>UNLICENSED</license>
  <buildtool_depend>ament_cmake</buildtool_depend>
  <export><build_type>ament_cmake</build_type></export>
</package>
```

File 26

campus_environment/src/um6p_campus_description/worlds/campus.sdf

```
<?xml version='1.0' encoding='utf-8'?>
<sdf version="1.10">
  <world name="um6p_campus">
    <plugin filename="gz-sim-scene-broadcaster-system" name="gz::sim::systems::SceneBroadcaster"
    />
    <scene>
      <ambient>.65 .65 .65 1</ambient>
      <background>.55 .72 .86 1</background>
      <shadows>true</shadows>
    </scene>
    <light name="warm_sun" type="directional">
      <pose>0 0 20 0 0 0</pose>
      <diffuse>.95 .86 .72 1</diffuse>
      <specular>.2 .2 .2 1</specular>
      <direction>-.45 .5 -.85</direction>
      <cast_shadows>true</cast_shadows>
    </light>
    <include>
      <uri>model://courtyard</uri>
      <name>courtyard</name>
      <pose>0 0 0 0 0 0</pose>
    </include>
    <include>
      <uri>model://pergola</uri>
      <name>pergola</name>
      <pose>0 0 0 0 0 0</pose>
    </include>
    <include>
      <uri>model://campus_buildings</uri>
      <name>campus_buildings</name>
      <pose>0 12 0 0 0 0</pose>
    </include>
    <include>
      <uri>model://um6p_landmark</uri>
      <name>um6p_landmark</name>
      <pose>-7 0 0 0 0 0</pose>
    </include>
    <include>
      <uri>model://landscaping</uri>
      <name>landscaping</name>
      <pose>0 0 0 0 0 0</pose>
    </include>
    <include>
      <uri>model://uav_1</uri>
      <name>uav_1</name>
      <pose>-3.6 -8.2 0.12 0 0 0.3</pose>
    </include>
    <include>
      <uri>model://uav_2</uri>
      <name>uav_2</name>
      <pose>3.6 -8.2 0.12 0 0 -0.3</pose>
    </include>
  </world>
</sdf>
```

File 27

campus_environment/tests/verify_assets.py

```
#!/usr/bin/env python3
"""Check installed assets and rendering-only scope, without running Gazebo."""
```

```

import hashlib, json, math, os, subprocess
from pathlib import Path
import xml.etree.ElementTree as E
from ament_index_python.packages import get_package_share_directory

pkg=Path(get_package_share_directory('um6p_campus_description'))
models=pkg/'models'; world=pkg/'worlds/campus.sdf'
w=E.parse(world).getroot().find('world')
assert w is not None and w.attrib['name']=='um6p_campus'
assert [x.attrib['name'] for x in w.findall('plugin')]=='gz::sim::systems::SceneBroadcaster'
assert len(w.findall('include'))==7
names=[]; meshes=0; triangles=0
for inc in w.findall('include'):
    name=inc.findtext('name');names.append(name)
    assert inc.findtext('uri')==model+'/' +name
    sdf=models/name/'model.sdf'; model=E.parse(sdf).getroot().find('model')
    assert model.findtext('static')==True
    assert not model.findall('./plugin')+model.findall('./sensor')+model.findall('./joint')
    for uri in model.findall('./uri'):
        assert uri.text.startswith('model://')
        path=models/uri.text.removeprefix('model://')
        assert path.is_file(),path
        vertices=0; faces=[]
        for line in path.read_text().splitlines():
            if line.startswith('mtllib '):
                assert (path.parent/line.split(maxsplit=1)[1]).is_file(), 'Missing MTL'
            if line.startswith('v '):
                values=list(map(float,line.split()[1:]));assert len(values)==3 and all(math.isfinite(x) for x
                    in values);vertices+=1
            if line.startswith('f '):faces.append([int(x.split('/')[0]) for x in line.split()[1:]])
            assert vertices>0 and faces and all(len(f)==3 and all(1<=i<=vertices for i in f) for f in faces)
        meshes+=1;triangles+=len(faces)
env=dict(os.environ,SDF_PATH=str(models))
r=subprocess.run(['gz','sdf','-k',str(sdf)],env=env,capture_output=True,text=True,timeout=15)
assert r.returncode==0 and 'Valid.' in r.stdout and 'Error' not in r.stderr,(sdf,r.stdout,r.
    stderr)
r=subprocess.run(['gz','sdf','-k',str(world)],env=env,capture_output=True,text=True,timeout=15)
assert r.returncode==0 and 'Valid.' in r.stdout and 'Error' not in r.stderr,(r.stdout,r.stderr)
assert sorted(n for n in names if n.startswith('uav_'))=='uav_1','uav_2'
print(json.dumps({'status':'PASS','models':names,'meshes':meshes,'triangles':triangles,'
    SDF_validation':'world plus all 7 models','resolved_from':'installed package'},indent=2))

```

File 28

campus_environment/tests/verify_runtime.py

```

#!/usr/bin/env python3
"""Bounded installed workspace check; never requests control, step or unpause."""
import argparse, json, math, os, re, signal, subprocess, time
from pathlib import Path
import rclpy
from std_msgs.msg import String
from gz.transport13 import Node
from gz.msgs10.empty_pb2 import Empty
from gz.msgs10.scene_pb2 import Scene
from gz.msgs10.world_stats_pb2 import WorldStatistics
from gz.msgs10.stringmsg_pb2 import StringMsg

ROOT=Path(__file__).resolve().parents[2]
p=argparse.ArgumentParser(); p.add_argument('--gui',action='store_true')
p.add_argument('--view',default='courtyard',choices=['courtyard','overview'])
p.add_argument('--output',default='evidence/runtime'); args=p.parse_args()
out=ROOT/args.output; out.mkdir(parents=True,exist_ok=True)
os.environ['GZ_PARTITION']='campus_verify_'+str(os.getpid())

```

```

os.environ['ROS_DOMAIN_ID']='84'
os.environ['ROS_AUTOMATIC_DISCOVERY_RANGE']='LOCALHOST'
os.environ['ROS_LOG_DIR']=str(ROOT/'log/verification')
result={'status':'FAIL','mode':'gui' if args.gui else 'headless','runs':[]}
stats=[]; gz=Node()
gz.subscribe(WorldStatistics,'/world/um6p_campus/stats',lambda m: stats.append({
    'paused':m.paused,'iterations':m.iterations,'sim_sec':m.sim_time.sec,
    'sim_nsec':m.sim_time.nsec,'stepping':m.stepping}))
rclpy.init(); node=rclpy.create_node('campus_verifier'); messages=[]
node.create_subscription(String,'/campus/evidence',lambda m:messages.append(m.data),10)
pub=gz.advertise('/campus/evidence',StringMsg)
try:
    for run in range(2):
        stats.clear(); messages.clear()
        log=open(out/f'launch-{run+1}.log','w')
        proc=subprocess.Popen([str(ROOT/'scripts/launch_paused.sh'),'gui' if args.gui else 'headless',
            args.view],stdout=log,stderr=subprocess.STDOUT,start_new_session=True)
        record={'pid':proc.pid}; result['runs'].append(record)
        try:
            deadline=time.monotonic()+35; scene=None
            while time.monotonic()<deadline:
                assert proc.poll() is None, 'launch exited before scene response'
                ok, reply=gz.request('/world/um6p_campus/scene/info',Empty(),Empty,Scene,1000)
                if ok and len(reply.model)==7: scene=reply;break
                time.sleep(.2)
            assert scene is not None, 'scene service did not return seven models'
            (out/f'scene-{run+1}.txt').write_text(str(scene))
            names=sorted(m.name for m in scene.model)
            assert names==sorted(['courtyard','pergola','campus_buildings','um6p_landmark','landscaping','uav_1','uav_2']), names
            poses={}
            for m in scene.model:
                if m.name.startswith('uav_'):
                    expected=(-3.6,-8.2,.12) if m.name=='uav_1' else (3.6,-8.2,.12)
                    actual=(m.pose.position.x,m.pose.position.y,m.pose.position.z)
                    assert all(abs(a-b)<1e-8 for a,b in zip(actual,expected)),actual
                    yaw=.3 if m.name=='uav_1' else -.3
                    assert abs(m.pose.orientation.z-math.sin(yaw/2))<1e-8
                    assert abs(m.pose.orientation.w-math.cos(yaw/2))<1e-8
                    poses[m.name]=list(actual)+[0,0,yaw]
            token=f'campus-paused-check-{run+1}'
            deadline=time.monotonic()+12
            while token not in messages and time.monotonic()<deadline:
                pub.publish(StringMsg(data=token));rclpy.spin_once(node,timeout_sec=.2)
            assert token in messages, 'No Gazebo-to-ROS text message exchange'
            # Wall time, independent of simulation time. No /clock assumption.
            deadline=time.monotonic()+(12 if args.gui else 4)
            while time.monotonic()<deadline:rclpy.spin_once(node,timeout_sec=.1)
            assert len(stats)>=3,'No sufficient world statistics samples'
            assert all(s['paused'] and s['iterations']==0 and s['sim_sec']==0 and s['sim_nsec']==0 and not
                s['stepping'] for s in stats),stats[-3:]
            discovered=sorted(node.get_node_names())
            assert discovered==['campus_evidence_bridge','campus_verifier'],discovered
            record.update(names=names,uav_poses=poses,ros_nodes=discovered,bridge_received=token,
                stats_samples=len(stats),stats_first=stats[0],stats_last=stats[-1])
            if args.gui and run==0:
                services=subprocess.run(['gz','service','-l'],capture_output=True,text=True,timeout=5)
                (out/'gui-services.txt').write_text(services.stdout)
                # Screenshot service is an inspection-only GUI operation.
                assert '/gui/screenshot' in services.stdout, 'Screenshot service unavailable'
                if '/gui/screenshot' in services.stdout:
                    from gz.msgs10.boolean_pb2 import Boolean
                    from gz.msgs10.gui_camera_pb2 import GUICamera
                    from ament_index_python.packages import get_package_share_directory
                    cfg=Path(get_package_share_directory('um6p_campus_bringup'))/'config'/'overview.config' if

```

```

args.view=='overview' else 'gui.config')
xyzrpy=list(map(float,re.search(r'<camera_pose>(.*?)</camera_pose>',cfg.read_text()).group(1)
.split()))
camera=GUICamera();camera.pose.position.x,camera.pose.position.y,camera.pose.position.z=
xyzrpy[:3]
pitch,yaw=xyzrpy[4:]
camera.pose.orientation.x=-math.sin(yaw/2)*math.sin(pitch/2)
camera.pose.orientation.y=math.cos(yaw/2)*math.sin(pitch/2)
camera.pose.orientation.z=math.sin(yaw/2)*math.cos(pitch/2)
camera.pose.orientation.w=math.cos(yaw/2)*math.cos(pitch/2)
command=['gz','service','-s','/gui/move_to/pose','--reqtype','gz.msgs.GUICamera','--reptype
','gz.msgs.Boolean','--timeout','5000','--req',str(camera)]
moved=subprocess.run(command,capture_output=True,text=True,timeout=8)
record['camera_reset_response']=moved.stdout
assert moved.returncode==0 and 'data: true' in moved.stdout, 'Saved camera reset failed: '+
moved.stdout+moved.stderr
time.sleep(2)
record['screenshot_camera_xyzrpy']=xyzrpy
before=set(out.glob('*.png'))
shot=subprocess.run(['gz','service','-s','/gui/screenshot','--reqtype','gz.msgs.StringMsg
','--reptype','gz.msgs.Boolean','--timeout','5000','--req',str(StringMsg(data=str(out.resolve
())))],capture_output=True,text=True,timeout=8)
record['screenshot_service']={'exit_code':shot.returncode,'reply':shot.stdout}
assert shot.returncode==0 and 'data: true' in shot.stdout, 'Screenshot request failed: '+shot
.stdout+shot.stderr
deadline=time.monotonic()+5
while not (set(out.glob('*.png'))-before) and time.monotonic()<deadline:time.sleep(.1)
assert set(out.glob('*.png'))-before, 'No actual screenshot file saved'
record['screenshot_files']=[p.name for p in set(out.glob('*.png'))-before]
assert all(s['paused'] and s['iterations']==0 and s['sim_sec']==0 and s['sim_nsec']==0 for s
in stats), 'Time advanced during camera inspection'
record['status']='PASS'
finally:
if proc.poll() is None:proc.send_signal(signal.SIGINT)
try:proc.wait(timeout=15)
except subprocess.TimeoutExpired:
os.killpg(proc.pid,signal.SIGTERM);proc.wait(timeout=5)
record['forced_shutdown']=True
log.close();record['exit_code']=proc.returncode
text=(out/f'launch-{run+1}.log').read_text()
assert 'process has died' not in text, 'Gazebo or bridge child failed; inspect launch log'
assert 'Missing material' not in text and 'Unable to find uri' not in text, 'Resource warnings'
assert '[Err]' not in text, 'Gazebo error in launch log'
assert proc.returncode==0, f'unclean launch exit {proc.returncode}'
time.sleep(1)
result['status']='PASS'
except Exception as exc:
result['error']=str(exc); raise
finally:
result['stats_tail']=stats[-5:]
(out/'results.json').write_text(json.dumps(result,indent=2)+'\n')
node.destroy_node();rclpy.shutdown()
print(json.dumps(result,indent=2))

```

File 29

references/demo-packages.txt

```

# Additional/direct C++ development dependencies; existing stack already supplies them.
ros-jazzy-gz-sim-vendor
ros-jazzy-gz-plugin-vendor
ros-jazzy-ros-gz-bridge
ros-jazzy-rosgraph-msgs
ros-jazzy-std-msgs

```

```
build-essential
cmake
ros-jazzy-gz-gui-vendor
qtbase5-dev
qtdeclarative5-dev
qtquickcontrols2-5-dev
qml-module-qtquick-controls2
```

File 30

references/documentation-packages.txt

```
# Optional tutorial compilation and page inspection (Ubuntu apt packages).
texlive-latex-recommended
texlive-latex-extra
texlive-fonts-recommended
latexmk
poppler-utils
python3-pil
```

File 31

references/native-packages.txt

```
# ROS repository must already be configured, as on the user's laptop.
ros-jazzy-ros-base
ros-jazzy-ros-gz
ros-jazzy-ament-cmake
python3-colcon-common-extensions
python3-protobuf
python3-yaml
build-essential
cmake
```

File 32

references/observed-direct-package-pins.txt

```
build-essential=12.10ubuntu1
cmake=3.28.3-1build7
python3-colcon-common-extensions=0.3.0-100
python3-protobuf=3.21.12-8.2ubuntu0.3
python3-yaml=6.0.1-2build2
ros-jazzy-ament-cmake=2.5.6-2noble.20260225.222913
ros-jazzy-gz-msgs-vendor=0.0.8-1noble.20260905.033532
ros-jazzy-gz-rendering-vendor=0.0.7-1noble.20260905.065113
ros-jazzy-gz-sim-vendor=0.0.13-1noble.20260905.073557
ros-jazzy-gz-transport-vendor=0.0.9-1noble.20260905.065154
ros-jazzy-ros-base=0.11.0-1noble.20260903.093452
ros-jazzy-ros-gz=1.0.24-1noble.20260905.090616
ros-jazzy-ros-gz-bridge=1.0.24-1noble.20260905.070149
```

File 33

scripts/build.sh

```
#!/usr/bin/env bash
set -eo pipefail
ROOT="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")/.." && pwd)"
source /opt/ros/jazzy/setup.bash
cd "$ROOT"
```

```
colcon --log-base "$ROOT/log" build --base-paths "$ROOT/campus_environment/src" "$ROOT/common/src" \
  \
  "$ROOT/demo/demo_1_no_obstacle/src" "$ROOT/demo/demo_2_static_sphere/src" \
--build-base "$ROOT/build" --install-base "$ROOT/install" \
--event-handlers console_direct+
```

File 34

scripts/build_tutorial.sh

```
#!/usr/bin/env bash
exec bash "$(dirname -- "${BASH_SOURCE[0]}")/build_tutorials.sh" campus
```

File 35

scripts/build_tutorials.sh

```
#!/usr/bin/env bash
set -eo pipefail
ROOT="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")/.." && pwd)"
MODE="${1:-all}"
case "$MODE" in all|campus|demos) ;; *) echo 'Usage: build_tutorials.sh [all|campus|demos]' >&2;
  exit 2 ;; esac
python3 "$ROOT/scripts/tutorial_listings.py"
python3 "$ROOT/scripts/kinematic_listings.py"
for kind in campus_environment kinematic_demos; do
  [[ "$MODE" != campus || "$kind" == campus_environment ]] || continue
  [[ "$MODE" != demos || "$kind" == kinematic_demos ]] || continue
  cd "$ROOT/tutorial/source/$kind"
  if command -v latexmk >/dev/null; then
    latexmk -pdf -interaction=nonstopmode -halt-on-error tutorial.tex
  elif command -v tectonic >/dev/null; then
    tectonic --keep-logs tutorial.tex
  else
    echo 'Install optional documentation tools or provide Tectonic on PATH.' >&2; exit 2
  fi
  if [[ "$kind" == campus_environment ]]; then
    NAME=01_UM6P_Campus_Environment_Setup_and_Coding.pdf
  else
    NAME=02_Two_UAV_Kinematic_Demos_Coding_and_Verification.pdf
  fi
  cp tutorial.pdf "$ROOT/tutorial/$NAME"
  pdfinfo "$ROOT/tutorial/$NAME"
done
```

File 36

scripts/check_environment.sh

```
#!/usr/bin/env bash
set -eo pipefail
cat /etc/os-release
uname -m
df -h .
source /opt/ros/jazzy/setup.bash
command -v ros2 gz colcon
gz sim --versions
ros2 pkg prefix ros_gz_bridge
python3 -c 'import rclpy; print("rclpy: available")'
dpkg-query -W -f='${Package}\t${Version}\n' ros-jazzy-ros-gz \
  ros-jazzy-ros-gz-bridge ros-jazzy-gz-sim-vendor python3-colcon-common-extensions
```

File 37

scripts/launch_environment.sh

```
#!/usr/bin/env bash
set -eo pipefail
ROOT="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")/.." && pwd)"
source /opt/ros/jazzy/setup.bash
source "$ROOT/install/setup.bash"
export ROS_DOMAIN_ID="${ROS_DOMAIN_ID:-84}"
export ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST
export GZ_PARTITION="${GZ_PARTITION:-ccbo_campus}"
export ROS_LOG_DIR="$ROOT/log/ros"
export GZ_HOMEDIR="$ROOT/.runtime/gz"
mkdir -p "$ROS_LOG_DIR" "$GZ_HOMEDIR"
# Only the documented GUI selector is accepted; arbitrary run arguments cannot leak in.
VIEW="${2:-courtyard}"
case "$VIEW" in courtyard|overview) ;; *) echo "View must be courtyard or overview" >&2; exit 2 ;;
esac
case "${1:-gui}" in
gui) exec ros2 launch um6p_campus_bringup campus.launch.py gui:=true view:="$VIEW" ;;
headless) exec ros2 launch um6p_campus_bringup campus.launch.py gui:=false view:="$VIEW" ;;
*) echo 'Usage: launch_paused.sh [gui|headless] [courtyard|overview]' >&2; exit 2 ;;
esac
```

File 38

scripts/launch_paused.sh

```
#!/usr/bin/env bash
# Compatibility alias; use launch_environment.sh in new instructions.
exec bash "$(dirname -- "${BASH_SOURCE[0]}")/launch_environment.sh" "$@"
```

File 39

scripts/reproduce_clean.sh

```
#!/usr/bin/env bash
# Fresh build and bounded tests. Never overwrite a previous destination.
set -eo pipefail
ROOT="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")/.." && pwd)"
TARGET="${1:?Pass a new target directory (spaces supported)}"
[[ ! -e "$TARGET" ]] || { echo 'Target already exists; choose a fresh directory.' >&2; exit 2; }
TARGET="$(realpath -m -- "$TARGET")"
case "$TARGET/" in "$ROOT/"*) echo 'Target must be outside source folder.' >&2; exit 2 ;; esac
mkdir -p "$TARGET"
for item in campus_environment common demo scripts references docs .vscode .gitignore; do
cp -a -- "$ROOT/$item" "$TARGET/"
done
mkdir -p "$TARGET/evidence"
bash "$TARGET/scripts/build.sh" > "$TARGET/evidence/build.log" 2>&1
bash "$TARGET/scripts/verify_unit.sh" > "$TARGET/evidence/unit.log" 2>&1
bash "$TARGET/scripts/verify.sh" headless > "$TARGET/evidence/verification.log" 2>&1
for wave in 1 2; do
bash "$TARGET/scripts/verify_demo.sh" "$wave" headless "$TARGET/evidence/wave$wave" > "$TARGET/evidence/wave$wave.log" 2>&1
done
printf 'Fresh build, paused campus and both kinematic demos PASS: %s\n' "$TARGET"
```

File 40

scripts/setup.sh

```
#!/usr/bin/env bash
# One dependency plan for campus and both demos; installs only with explicit option.
exec bash "$(dirname -- "${BASH_SOURCE[0]}")/setup_demo.sh" "$@"
```

File 41

scripts/setup_demo.sh

```
#!/usr/bin/env bash
# Additive dependency check; no installation without explicit --install-missing.
set -eo pipefail
ROOT="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")/.." && pwd)"
MODE="${1:---plan}"
case "$MODE" in --plan|--install-missing) ;; *) echo 'Usage: setup_demo.sh [--plan|--install-missing]' >&2; exit 2 ;; esac
bash "$ROOT/scripts/setup_native.sh" "$MODE"
MISSING=()
while IFS= read -r package; do
    [[ -z "$package" || "$package" == \#* ]] && continue
    if [[ "$(dpkg-query -W -f='${db:Status-Status}' "$package" 2>/dev/null || true)" != installed
        ]]; then MISSING+=("$package"); fi
done < "$ROOT/references/demo-packages.txt"
if (( ${#MISSING[@]} == 0 )); then echo 'All demo packages installed; no changes.'; exit 0; fi
printf 'Missing demo package: %s\n' "${MISSING[@]}"
if [[ "$MODE" == --install-missing ]]; then
    sudo apt-get update
    sudo apt-get install --no-upgrade "${MISSING[@]}"
else
    echo 'Review these packages; --install-missing installs only missing dependencies.'
fi
```

File 42

scripts/setup_native.sh

```
#!/usr/bin/env bash
# Inspect first. Only --install-missing invokes apt; installed packages are not upgraded.
set -eo pipefail
ROOT="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")/.." && pwd)"
source /etc/os-release
[[ "$ID" == ubuntu && "$VERSION_ID" == 24.04 ]] || { echo 'Requires Ubuntu 24.04'; exit 2; }
[[ "$(dpkg --print-architecture)" == amd64 ]] || { echo 'This tested manifest targets amd64'; exit 2; }
MISSING=()
while IFS= read -r package; do
    [[ -z "$package" || "$package" == \#* ]] && continue
    if [[ "$(dpkg-query -W -f='${db:Status-Status}' "$package" 2>/dev/null || true)" != installed
        ]]; then MISSING+=("$package"); fi
done < "$ROOT/references/native-packages.txt"
if (( ${#MISSING[@]} == 0 )); then echo 'All required packages already installed; no changes.';
    exit 0; fi
printf 'Missing: %s\n' "${MISSING[@]}"
case "${1:---plan}" in
    --plan) echo 'Configure the official ROS Jazzy apt repository if absent; then rerun with --
        install-missing.' ;;
    --install-missing) sudo apt-get update; sudo apt-get install --no-upgrade "${MISSING[@]}" ;;
    *) echo 'Usage: setup_native.sh [--plan|--install-missing]' >&2; exit 2 ;;
esac
```

File 43

scripts/tutorial_listings.py

```
#!/usr/bin/env python3
"""Import complete maintained source files into LaTeX without copying their contents."""
from pathlib import Path
r=Path(__file__).resolve().parents[1]
files=[]
for folder in ['campus_environment/src','campus_environment/scripts','campus_environment/tests','vscode']:
    files += [p for p in (r/folder).rglob('*') if p.is_file() and (p.suffix in ('.py','.sh','.xml','.sdf','.config','.yaml','.json') or p.name=='CMakeLists.txt') and '__pycache__' not in p.parts]
files += [r/'scripts'/n for n in ['build.sh','check_environment.sh','setup.sh','setup_demo.sh','setup_native.sh','launch_environment.sh','launch_paused.sh','verify.sh','reproduce_clean.sh','build_tutorial.sh','build_tutorials.sh','tutorial_listings.py']]
files += [r/'references'/n for n in ['native-packages.txt','demo-packages.txt','documentation-packages.txt','observed-direct-package-pins.txt']]
text='\section{Complete first-party source listings}\n'
text+='These listings import the delivered files directly. Generated mesh coordinates are represented by the complete generator and separately hashed assets.\par\n'
for i,p in enumerate(sorted(set(files)),1):
    rel=p.relative_to(r).as_posix()
    text+='\subsection*{File '+str(i)+'}\n\nnoindent{\small\path{' +rel+'}}\par\n'
    text+='\lstinputlisting{../../../../'+rel+'}\n'
(r/'tutorial/source/campus_environment/listings.tex').write_text(text)
print('Imported',len(set(files)), 'source files for campus_environment')
```

File 44

scripts/verify.sh

```
#!/usr/bin/env bash
set -eo pipefail
ROOT="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")/.." && pwd)"
source /opt/ros/jazzy/setup.bash
source "$ROOT/install/setup.bash"
python3 "$ROOT/campus_environment/tests/verify_assets.py"
case "${1:-headless}" in
    headless) timeout --signal=INT --kill-after=15s 120s python3 "$ROOT/campus_environment/tests/verify_runtime.py" --output evidence/local-headless ;;
    gui) timeout --signal=INT --kill-after=15s 120s python3 "$ROOT/campus_environment/tests/verify_runtime.py" --gui --output evidence/local-gui ;;
    assets) ;;
    *) echo 'Usage: verify.sh [assets|headless|gui]' >&2; exit 2 ;;
esac
```